

Unidad 12: Despliegue y CI/CD

Post-Contenido 2 — Proyecto: Pipeline CI/CD con GitHub Actions y Docker Hub

Sergio Andrés Moreno Sánchez

02230131001

Jonathan Rolando Rey Castillo

Universidad de Santander

UDES – Campus Cúcuta

Cúcuta, Norte de Santander

2026

Información General

Imagen Docker Hub: <https://hub.docker.com/r/sergiomorenos/mi-spring-app>

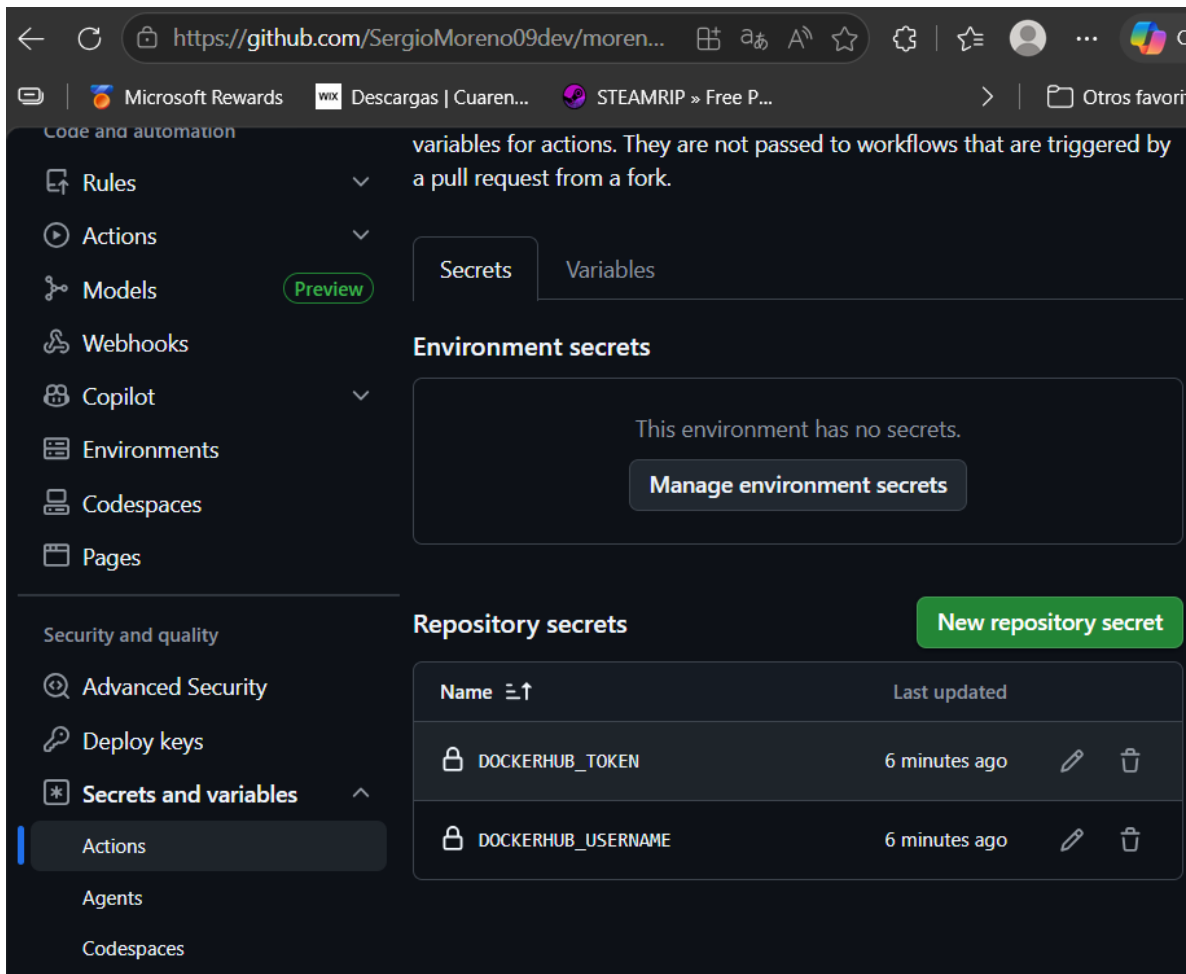
Tecnologías utilizadas: Spring Boot 4.0.5 · GitHub Actions · Docker · Docker Hub · JaCoCo · Maven

Configuración de GitHub Secrets

Para autenticarse en Docker Hub desde el runner de GitHub Actions sin exponer credenciales en el código, se configuraron dos secrets en el repositorio desde Settings en la sección de secrets and variables:

Secret	Descripción
DOCKERHUB_USERNAME	Nombre de usuario de Docker Hub
DOCKERHUB_TOKEN	Access Token generado en Docker Hub con permisos de lectura y escritura

El Access Token se generó desde hub.docker.com en la sección de security. Los secrets se referencian en el archivo YAML como `${{ secrets.DOCKERHUB_USERNAME }}` y `${{ secrets.DOCKERHUB_TOKEN }}` sin exponer sus valores en ningún archivo del repositorio.



Workflow de GitHub Actions

Se creó el archivo `.github/workflows/ci.yml` en la raíz del repositorio. El pipeline se activa automáticamente en cada push a la rama main y en pull requests. Contiene dos jobs encadenados:

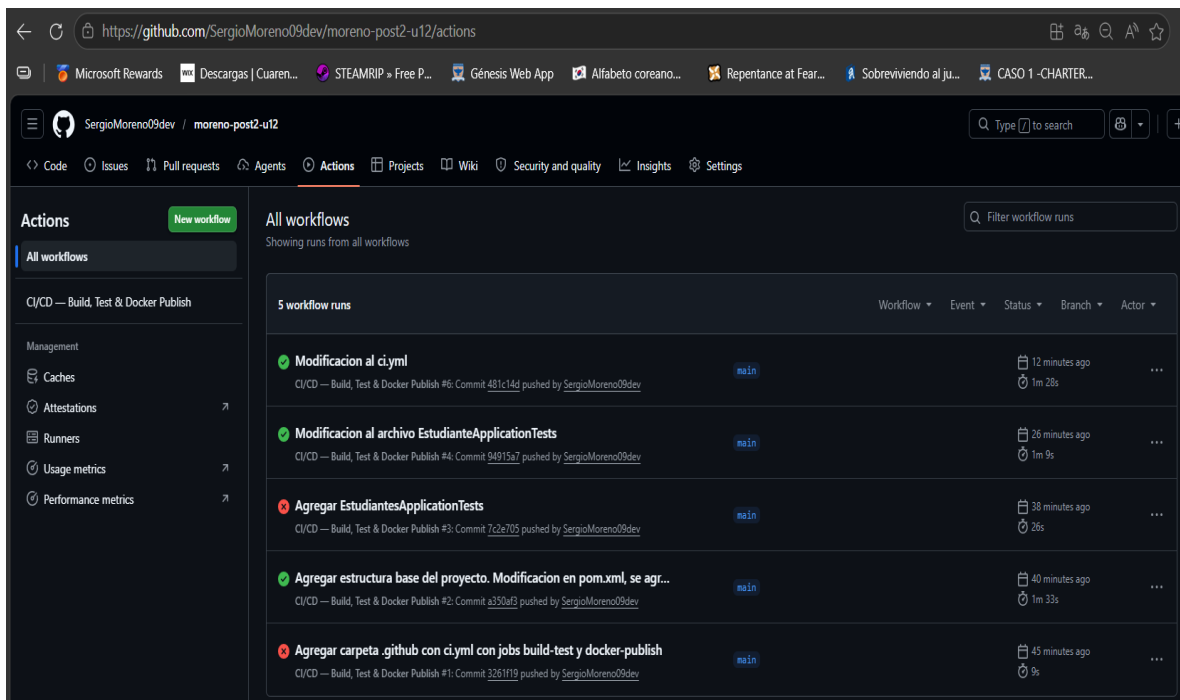
Job 1 — build-and-test (CI):

Se ejecuta en un runner ubuntu-latest. Realiza checkout del código, configura JDK 21 con caché de dependencias Maven para acelerar builds posteriores, ejecuta `mvn clean verify` que compila el proyecto, corre las pruebas unitarias y genera el reporte de cobertura JaCoCo. El reporte se sube como artefacto descargable con retención de 7 días usando `actions/upload-artifact@v4`.

Las pruebas unitarias implementadas verifican la lógica central del proyecto: que el helper method `agregarEstudiante()` sincroniza correctamente la relación bidireccional entre `Curso` y `Estudiante` en ambos sentidos, y que `quitarEstudiante()` elimina correctamente la asociación en ambas entidades sin dejar datos huérfanos.

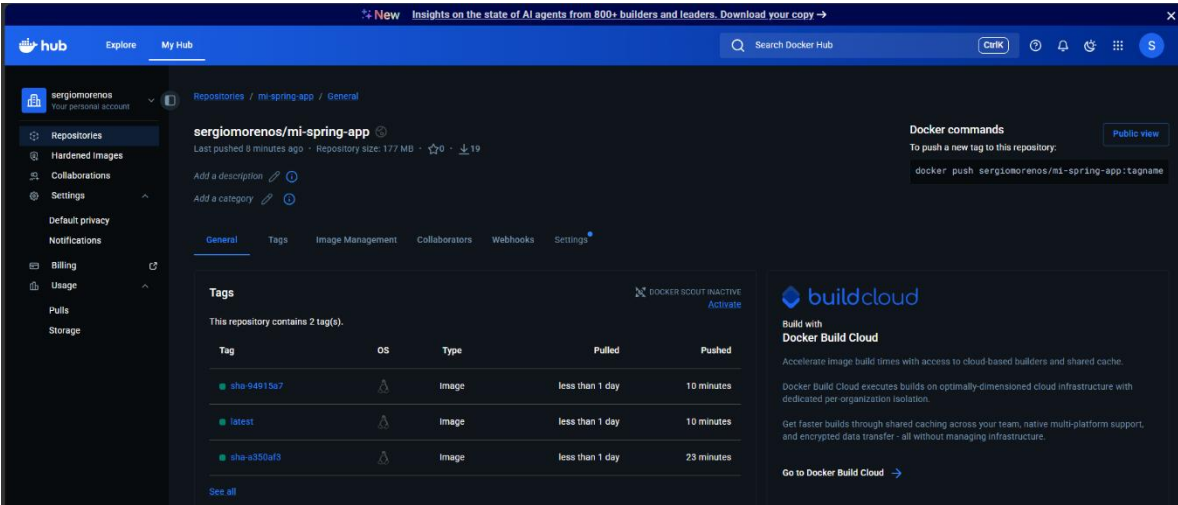
Job 2 — docker-publish (CD):

Solo se ejecuta cuando el Job 1 completa exitosa y únicamente en push a main, no en pull requests. Realiza checkout del código, se autentica en Docker Hub usando los secrets configurados, extrae metadata para generar los tags de la imagen usando docker/metadata-action@v5, y construye y publica la imagen usando docker/build-push-action@v5 con el Dockerfile multi-stage del proyecto.



Verificación del Pipeline Completo

Después de hacer push a main con el archivo ci.yml, el pipeline se ejecutó automáticamente. Ambos jobs completaron con check verde. La imagen fue publicada en Docker Hub con los tags latest y sha-<commit> correspondiente al último push.



Pruebas Unitarias Implementadas

Se implementaron dos pruebas unitarias en `EstudiantesApplicationTests.java` que validan la lógica de negocio del dominio sin depender de base de datos ni contexto de Spring:

Prueba	Descripción
agregarEstudianteSincronizaCursoYEstudiante	Crea un Curso y un Estudiante, llama a <code>agregarEstudiante()</code> y verifica que el estudiante queda en la colección del curso y que el curso queda en la colección del estudiante
quitarEstudianteSincronizaCursoYEstudiante	Agrega un estudiante a un curso y luego llama a <code>quitarEstudiante()</code> , verificando que ambas colecciones quedan vacías

Ambas pruebas validan directamente los helper methods de la entidad `Curso`, que son responsables de mantener sincronizada la relación bidireccional `@ManyToMany` entre `Curso` y `Estudiante`. Al no usar `@SpringBootTest` ni conexión a base de datos, las pruebas se ejecutan de forma rápida y confiable en cualquier entorno, incluyendo el runner de GitHub Actions.

Conclusiones

Se implementó exitosamente un pipeline CI/CD completo con GitHub Actions que automatiza la compilación, ejecución de pruebas, generación de reporte de cobertura JaCoCo y publicación de la imagen Docker en Docker Hub en cada push a la rama

principal. El encadenamiento de jobs con needs: garantiza que la imagen solo se publica si las pruebas pasan exitosamente. El uso de GitHub Secrets protege las credenciales de Docker Hub sin exponerlas en el código. El etiquetado semántico de la imagen con latest y el SHA del commit permite rastrear exactamente qué versión del código corresponde a cada imagen publicada.